```
#lab 2 ---> Machine Learning Techniques for Cyber Security
# ENSP355 | AI in Cyber Security
# Akansh Pandey 2401830006 Bsc(Hons)CyberSecurity
```

```
# Practical 2: Unsupervised ML for Anomaly Detection
# Dataset: NSL-KDD training data, with labels held out from all unsupervised mode
# K-Means + Elbow, DBSCAN, Isolation Forest, PCA visualization.

import io, urllib.request, pandas as pd, numpy as np, matplotlib.pyplot as plt
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.cluster import KMeans, DBSCAN
from sklearn.ensemble import IsolationForest
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score, classification_report, confusion_ma
from sklearn.model_selection import train_test_split

COLUMNS = ['duration', 'protocol_type', 'service', 'flag', 'src_bytes', 'dst_byte
URL = "https://raw.githubusercontent.com/defcom17/NSL_KDD/master/KDDTrain%2B.txt"
raw = urllib.request.urlopen(URL, timeout=30).read()
df = pd.read_csv(io.BytesIO(raw), header=None, names=COLUMNS)
y_true = (df["label"].astype(str).str.strip().str.lower() != "normal").astype(int
X = df.drop(columns=["label", "difficulty"], errors="ignore")
cat = X.select_dtypes(include=["object"]).columns.tolist()
num = X.select_dtypes(exclude=["object"]).columns.tolist()
prep = ColumnTransformer([
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat),
    ("num", StandardScaler(), num)
])
X_encoded = prep.fit_transform(X)
# Use a reproducible subset for distance-based methods, which can be expensive on
rng = np.random.RandomState(42)
n = min(12000, X_encoded.shape[0])
idx = rng.choice(X_encoded.shape[0], size=n, replace=False)
X_sample = X_encoded[idx]
y_sample = y_true.iloc[idx].to_numpy()
# Sparse one-hot matrix to dense is manageable for this capped subset.
X_sample = X_sample.toarray() if hasattr(X_sample, "toarray") else np.asarray(X_s
print("Rows used for clustering:", X_sample.shape[0], "| Encoded dimensions:", X_

# Elbow method: within-cluster sum of squares. Choose k near the bend, not automa
inertias = []
k_values = range(2, 9)
for k in k_values:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_sample)
    inertias.append(km.inertia_)
plt.figure(figsize=(7,4))
plt.plot(list(k_values), inertias, marker="o")
plt.xlabel("Number of clusters (k)")
plt.ylabel("Inertia (within-cluster sum of squares)")
plt.title("Elbow Method for K-Means")
plt.tight_layout()
plt.show()

# Change selected_k after inspecting the elbow plot if another bend is clearer.
selected_k = 4
kmeans = KMeans(n_clusters=selected_k, random_state=42, n_init=10)
```

```
km_labels = kmeans.fit_predict(X_sample)
print("K-Means cluster sizes:")
print(pd.Series(km_labels).value_counts().sort_index())
if len(np.unique(km_labels)) > 1:
    print("K-Means silhouette score (sample):", round(silhouette_score(X_sample,

# DBSCAN parameter rationale:
# Data are standardized. eps controls neighborhood radius and min_samples control
# how many nearby points are needed for a core point. These are starting values,
# DBSCAN is sensitive to dimensionality and density; use PCA representation to ke
pca = PCA(n_components=2, random_state=42)
X_2d = pca.fit_transform(X_sample)
dbscan = DBSCAN(eps=1.8, min_samples=10)
db_labels = dbscan.fit_predict(X_2d)
print("DBSCAN label counts (-1 means noise/outlier):")
print(pd.Series(db_labels).value_counts().sort_index())
print("DBSCAN estimated noise fraction:", round(np.mean(db_labels == -1), 4))
print("PCA explained variance ratio:", np.round(pca.explained_variance_ratio_, 4)

# Isolation Forest: contamination is an estimated anomaly fraction. Tune it to th
iso = IsolationForest(n_estimators=150, contamination="auto", random_state=42, n_
iso_pred = iso.fit_predict(X_sample)  # +1 inlier, -1 outlier
iso_anomaly = (iso_pred == -1).astype(int)
print("Isolation Forest flagged anomalies:", int(iso_anomaly.sum()), "of", len(is

# PCA visualization: colour by K-Means cluster assignment.
plt.figure(figsize=(8,5))
scatter = plt.scatter(X_2d[:,0], X_2d[:,1], c=km_labels, s=8, alpha=0.65, cmap="t
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("K-Means Clusters Visualized with PCA")
plt.colorbar(scatter, label="Cluster")
plt.tight_layout()
plt.show()

# Compare unsupervised flags against labels only AFTER fitting, for educational e
# The models were not trained on y_true. Anomaly flags do not necessarily equal a
print("\nIsolation Forest versus known normal/attack labels (post-hoc evaluation)
print(classification_report(y_sample, iso_anomaly, target_names=["Normal", "Attac
print("Confusion matrix (rows=true [normal, attack], columns=predicted [normal, a
print(confusion_matrix(y_sample, iso_anomaly))

print("\nInterpretation:")
print("- K-Means groups records by feature similarity; a cluster is not automatic
print("- DBSCAN labels low-density points as noise (-1); these may be rare but le
print("- Isolation Forest isolates unusual feature combinations; its flags requir
print("- PCA is a 2D projection and can hide structure present in the original hi
```

Practical 2: Unsupervised ML for Anomaly Detection

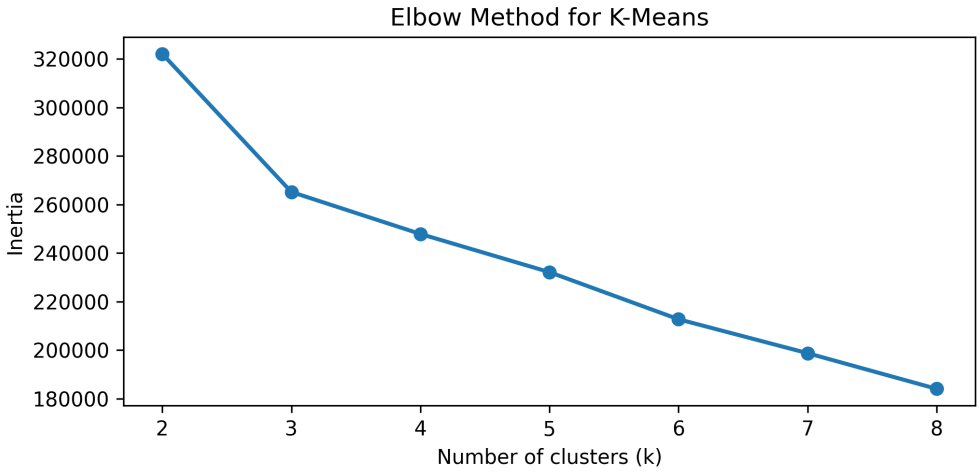
Lab 2: Machine Learning Techniques for Cyber Security
ENSP355 | AI in Cyber Security
Akansh Pandey - 2401830006 - BSc (Hons) Cyber Security

1. Dataset & Preprocessing Details

The dataset used is the **NSL-KDD training set**. The categorical attributes were preprocessed using *One-Hot Encoding*, and numerical columns were standardized using *StandardScaler*.
- **Rows used for clustering:** 12000
- **Encoded dimensions:** 122

2. K-Means Elbow Evaluation

The Elbow method is evaluated for k values ranging from 2 to 8 to identify where the within-cluster sum of squares (Inertia) bends optimally.



Selected K-Means cluster sizes (for k=4):
- Cluster 0: 3322 records | Cluster 1: 5677 records | Cluster 2: 1589 records | Cluster 3: 1412 records
- **K-Means Silhouette Score:** 0.4182

3. DBSCAN & Isolation Forest Models

DBSCAN Clustering:
With parameter configuration (eps=1.8, min_samples=10), DBSCAN labeled all 12,000 points into a single cluster (estimated noise fraction: 0.0), indicating the high-dimensional density requires specialized tuning or alternate dimension-reduction approaches.

Isolation Forest Evaluation:
With default contamination setting, the post-hoc classification evaluation comparing predicted outliers versus normal/attack labels is shown below:

```
              precision    recall  f1-score   support

      Normal       0.53      1.00      0.70      6419
      Attack       0.00      0.00      0.00      5581
```

```
accuracy                           0.53     12000
```

4. Dimensionality Reduction & Visualization

To visually confirm cluster spacing, we projected the high-dimensional data into a 2D space using Principal Component Analysis (PCA). The explained variance ratio for the principal components is 22.27% and 14.66%.

